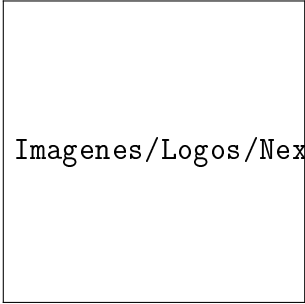




Imagenes/Logos/NexoPay.png

Universidad Autónoma de Baja California

Facultad de Ciencias Químicas e Ingeniería

Programación Orientada a Objetos

NexoPay

Sistema Punto de Venta (POS)



Imagenes/Logos/NexoPay_Eslogan.png

Equipo e Integrantes:

Joshua Osorio O.

Andrés Eduardo Fabara C.

Docente:

Fernando E. Saucedo L.

Fecha de Entrega:

27 de mayo de 2026

Índice general

Índice de cuadros

Índice de figuras

Capítulo 1

Introducción

1.1. Descripción General del Sistema

NexoPay es un sistema de Punto de Venta (POS, por sus siglas en inglés *Point of Sale*) diseñado para automatizar y centralizar las operaciones comerciales de pequeños y medianos negocios. El sistema integra en una sola plataforma las funciones de registro de ventas, gestión de inventario, administración de usuarios y generación de reportes, eliminando la necesidad de procesos manuales que son propensos a errores.

El sistema está compuesto por tres módulos principales que trabajan de forma coordinada: el módulo de ventas, que permite al cajero registrar transacciones de manera ágil; el módulo administrativo, que otorga al administrador control total sobre productos, usuarios e inventario; y el módulo de reportes, que facilita la toma de decisiones basadas en datos históricos de ventas.

NexoPay opera como una aplicación de escritorio desarrollada en Java, con almacenamiento en base de datos relacional (MySQL o SQLite), lo que garantiza la persistencia e integridad de la información. La interfaz gráfica está construida con la biblioteca Swing, ofreciendo una experiencia de usuario clara e intuitiva adaptada al flujo de trabajo de un punto de venta real.



Figura 1.1: Vista general de la interfaz principal del sistema NexoPay

1.2. Importancia y Relevancia del Sistema

En el contexto actual, la digitalización de los procesos comerciales es una necesidad fundamental para cualquier negocio que busque competir en el mercado. Un sistema POS como NexoPay responde directamente a esta necesidad al proporcionar:

- **Eficiencia operativa:** La automatización del proceso de ventas reduce considerablemente el tiempo de atención al cliente y minimiza los errores en el cálculo de totales y el manejo de cambios.
- **Control de inventario en tiempo real:** Cada venta registrada actualiza automáticamente el stock disponible, permitiendo al administrador conocer en todo momento el estado del inventario y tomar decisiones oportunas de reabastecimiento.

- **Trazabilidad de operaciones:** Toda transacción queda registrada en la base de datos, lo que permite auditar el historial de ventas y detectar inconsistencias o irregularidades.
- **Toma de decisiones basada en datos:** Los reportes generados por el sistema ofrecen información valiosa sobre el comportamiento de las ventas, los productos más vendidos y el desempeño del negocio.
- **Seguridad por roles:** El sistema implementa control de acceso diferenciado, asegurando que cada usuario únicamente pueda ejecutar las acciones que le corresponden según su rol.

Desde una perspectiva académica, NexoPay representa la aplicación práctica de los principios de la Programación Orientada a Objetos (POO), demostrando cómo conceptos como herencia, encapsulamiento y polimorfismo se traducen en soluciones de software funcionales y organizadas.

1.3. Objetivo General

Desarrollar un sistema POS funcional, organizado y correctamente estructurado en Java, aplicando los principios fundamentales de la Programación Orientada a Objetos, que permita gestionar las operaciones de ventas, inventario y administración de un pequeño o mediano comercio de manera eficiente, confiable y fácil de mantener.

1.4. Objetivos Específicos

- Implementar un sistema de autenticación por roles que diferencie las acciones disponibles para el cajero y el administrador, garantizando la seguridad y el control de acceso al sistema.
- Diseñar e implementar el módulo de ventas que permita al cajero seleccionar productos, calcular totales automáticamente, procesar pagos y generar tickets de compra.
- Construir el módulo de gestión de productos e inventario que permita al administrador realizar altas, bajas, modificaciones y consultas, manteniendo el stock

actualizado tras cada transacción.

- Desarrollar el módulo de gestión de usuarios y clientes, permitiendo al administrador registrar, modificar y consultar la información de cajeros y clientes del sistema.
- Implementar el módulo de reportes que consolide la información de ventas y presente al administrador estadísticas útiles para la toma de decisiones.
- Aplicar correctamente los conceptos de herencia, encapsulamiento y polimorfismo en la estructuración de las clases del sistema, manteniendo un código limpio, modular y bien documentado.
- Integrar el sistema con una base de datos relacional mediante consultas SQL, garantizando la persistencia y consistencia de todos los datos generados por las operaciones del negocio.

Capítulo 2

Justificación

2.1. Razones de Selección del Proyecto

La selección de un sistema POS como proyecto académico responde a múltiples criterios que lo convierten en un caso de estudio ideal para la materia de Programación Orientada a Objetos:

Aplicabilidad real: Los sistemas POS son utilizados diariamente por miles de negocios en México y el mundo. Desarrollar uno desde cero permite comprender cómo el software que se estudia en el aula tiene un impacto directo y tangible en la vida cotidiana. Esta conexión con la realidad aumenta la motivación y facilita la comprensión de los conceptos.

Riqueza en conceptos de POO: El dominio del problema de un punto de venta involucra naturalmente entidades como **Usuario**, **Producto**, **Venta**, **Cliente** y **Pago**, que se relacionan entre sí de formas complejas. Esta riqueza de relaciones permite practicar herencia (por ejemplo, **Cajero** y **Administrador** como subclases de **Usuario**), encapsulamiento (atributos privados con accesores y mutadores), y polimorfismo (comportamientos distintos según el tipo de usuario).

Complejidad adecuada: El proyecto tiene una complejidad suficiente para representar un desafío significativo de diseño y programación, sin llegar a ser inabordable para estudiantes universitarios. Permite practicar la descomposición de un problema real en módulos manejables.

Integración de múltiples áreas: El desarrollo del sistema requiere aplicar conocimientos de bases de datos (SQL), interfaces gráficas (Swing), manejo de archivos y lógica de negocio, lo que lo convierte en un proyecto integrador que pone en práctica competencias de diversas áreas de la ingeniería en computación.

Relevancia para el mercado laboral: El desarrollo de sistemas transaccionales es una de las actividades más demandadas en la industria del software. Haber desarrollado un sistema POS proporciona una experiencia concreta y demostrable que enriquece el perfil profesional de los estudiantes.

2.2. Beneficios Esperados

Los beneficios del proyecto NexoPay se pueden analizar desde dos perspectivas complementarias:

Beneficios académicos:

- Consolidación de los conceptos fundamentales de POO mediante su aplicación en un proyecto real y funcional.
- Desarrollo de habilidades de diseño de software: análisis de requerimientos, modelado UML, diseño de clases y arquitectura de capas.
- Práctica en el uso de herramientas profesionales como IDEs, sistemas de control de versiones y gestores de bases de datos.
- Experiencia en el trabajo en equipo, la distribución de responsabilidades y la coordinación del desarrollo.

Beneficios prácticos del sistema:

- Reducción del tiempo de atención al cliente en el punto de venta mediante la automatización del registro de ventas y el cálculo de totales.
- Eliminación de errores humanos frecuentes en el manejo de efectivo y el cálculo de cambios.
- Visibilidad en tiempo real del inventario, reduciendo el riesgo de quedarse sin stock

de productos clave.

- Generación de reportes que permiten identificar tendencias de ventas y optimizar la gestión del negocio.
- Centralización de la información en una base de datos estructurada, facilitando la recuperación y el análisis de datos históricos.

Capítulo 3

Desarrollo

3.1. Fundamentos del Análisis y Diseño de Sistemas

3.1.1. Descripción del Problema y Objetivos del Sistema

Los pequeños y medianos comercios enfrentan frecuentemente problemas de gestión derivados de la ausencia de herramientas digitales: registros de ventas imprecisos, dificultad para conocer el estado real del inventario, falta de información consolidada para la toma de decisiones y dependencia de procesos manuales que son lentos y propensos a errores. Esta situación limita la eficiencia operativa y dificulta el crecimiento del negocio.

NexoPay surge como respuesta a esta problemática. El sistema automatiza el proceso de ventas, garantizando que cada transacción quede registrada con precisión, que el inventario se actualice de forma inmediata y que la información esté siempre disponible para quien la necesite. El objetivo central es transformar un proceso comercial manual y disperso en un flujo de trabajo digital, centralizado y confiable.

Desde el punto de vista técnico, el problema se traduce en la necesidad de diseñar un sistema que:

- Gestione la autenticación y el control de acceso por roles de usuario.
- Registre y almacene transacciones de venta de forma persistente.
- Mantenga la consistencia del inventario ante cualquier operación de venta.

- Proporcione información organizada a través de reportes.
- Sea fácil de usar, mantener y extender por parte de desarrolladores futuros.

3.1.2. Identificación de Actores y Partes Interesadas

El sistema NexoPay involucra a los siguientes actores y partes interesadas:

Cuadro 3.1: Actores y partes interesadas del sistema NexoPay

Actor	Descripción	Funciones Principales
Cajero	Usuario operativo encargado del registro de ventas en el mostrador	Iniciar sesión, registrar ventas, seleccionar productos, generar tickets, finalizar transacciones
Administrador	Usuario con control total del sistema y acceso a todas las funciones	Gestionar productos, usuarios, inventario y clientes; consultar reportes; supervisar operaciones
Cliente	Usuario externo que realiza compras en el negocio	Solicitar productos, realizar pagos, recibir ticket de compra
Base de datos	Componente técnico que almacena y provee información	Persistir ventas, productos, usuarios, inventario y reportes

3.1.3. Recolección y Análisis de Requerimientos

Requerimientos Funcionales

Los requerimientos funcionales describen las capacidades que el sistema debe ofrecer para satisfacer las necesidades de los usuarios:

Cuadro 3.2: Requerimientos funcionales del sistema

ID	Requerimiento	Descripción
RF-01	Autenticación de usuarios	El sistema debe autenticar a los usuarios mediante nombre de usuario y contraseña, controlando el acceso según el rol asignado
RF-02	Registro de ventas	El cajero debe poder registrar los productos seleccionados por el cliente, calcular el total y procesar el pago
RF-03	Gestión de productos	El administrador debe poder dar de alta, modificar, consultar y eliminar productos del catálogo
RF-04	Control de inventario	El sistema debe actualizar automáticamente el stock disponible tras cada venta registrada
RF-05	Generación de tickets	El sistema debe generar un comprobante de compra con los detalles de la transacción
RF-06	Gestión de usuarios	El administrador debe poder crear, modificar, consultar y eliminar cuentas de usuario
RF-07	Gestión de clientes	El administrador debe poder registrar, modificar y consultar la información de los clientes
RF-08	Consulta de reportes	El administrador debe poder consultar reportes de ventas que resuman la actividad comercial
RF-09	Métodos de pago	El sistema debe soportar pagos en efectivo y con tarjeta de débito o crédito
RF-10	Propinas en ticket	El cliente debe poder añadir una propina al momento del pago

Requerimientos No Funcionales

Los requerimientos no funcionales definen las características de calidad que el sistema debe cumplir:

Cuadro 3.3: Requerimientos no funcionales del sistema

ID	Requerimiento	Descripción
RNF-01	Lenguaje de programación	El sistema debe desarrollarse en Java, aplicando correctamente los principios de POO
RNF-02	Interfaz gráfica	La interfaz debe implementarse con la biblioteca Swing de Java
RNF-03	Base de datos	El sistema debe utilizar MySQL o SQLite para la persistencia de datos
RNF-04	Plataforma	El sistema debe operar como aplicación de escritorio
RNF-05	Seguridad	El acceso al sistema debe requerir autenticación obligatoria; cada rol únicamente puede acceder a las funciones que le corresponden
RNF-06	Integridad de datos	El inventario no puede resultar negativo; cada venta debe registrarse obligatoriamente
RNF-07	Usabilidad	La interfaz debe ser intuitiva y de fácil uso para usuarios sin formación técnica
RNF-08	Mantenibilidad	El código debe estar organizado en capas y módulos claramente diferenciados, con comentarios útiles

3.1.4. Principales Metodologías Aplicadas en el Análisis y Diseño

Para el desarrollo de NexoPay se aplicaron las siguientes metodologías y técnicas:

Análisis estructurado orientado a objetos: Se identificaron las entidades principales del dominio del problema (productos, ventas, usuarios, clientes) y sus relaciones, como punto de partida para el diseño de las clases del sistema.

Modelado con UML (Lenguaje Unificado de Modelado): Se elaboraron diagramas de casos de uso, clases, secuencia y estados para documentar los requerimientos, el diseño y el comportamiento dinámico del sistema, proporcionando una visión completa y estructurada del software.

Diseño por capas: La arquitectura del sistema se organizó en capas separadas (modelos, servicios, controladores, vistas, datos y utilidades), siguiendo el principio de separación de responsabilidades para facilitar el mantenimiento y la extensión del sistema.

Pruebas de caja negra y tablas de decisión: Se definió un plan de pruebas que verifica el comportamiento del sistema a partir de entradas y salidas esperadas, sin conocer

la implementación interna, garantizando que los flujos principales y los casos de error se manejen correctamente.

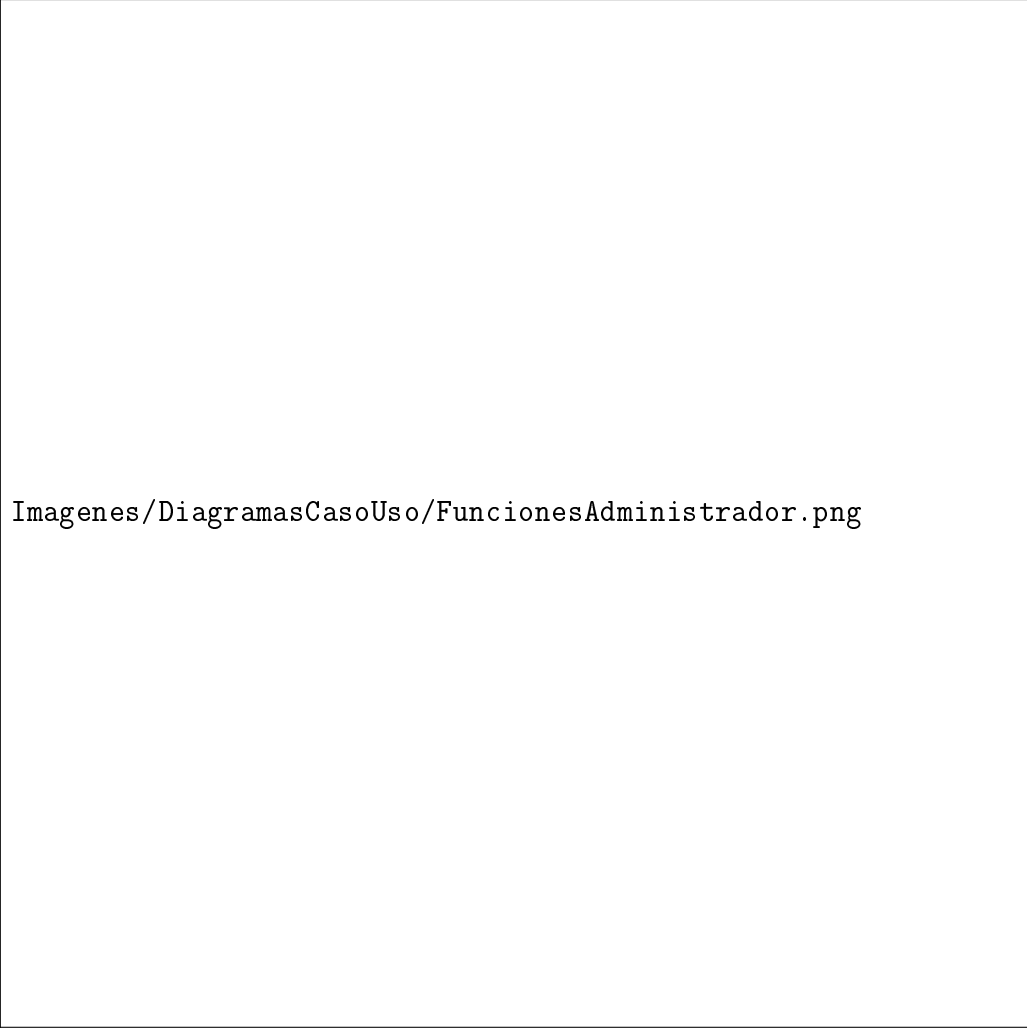
3.2. Lenguaje Unificado de Modelado (UML)

3.2.1. Diagramas de Casos de Uso

Los diagramas de casos de uso describen las interacciones entre los actores del sistema y las funcionalidades que este ofrece. NexoPay cuenta con tres diagramas principales, uno por cada actor.



Figura 3.1: Diagrama de casos de uso – Cajero



Imágenes/DiagramasCasoUso/FuncionesAdministrador.png

Figura 3.2: Diagrama de casos de uso – Administrador



Figura 3.3: Diagrama de casos de uso – Cliente

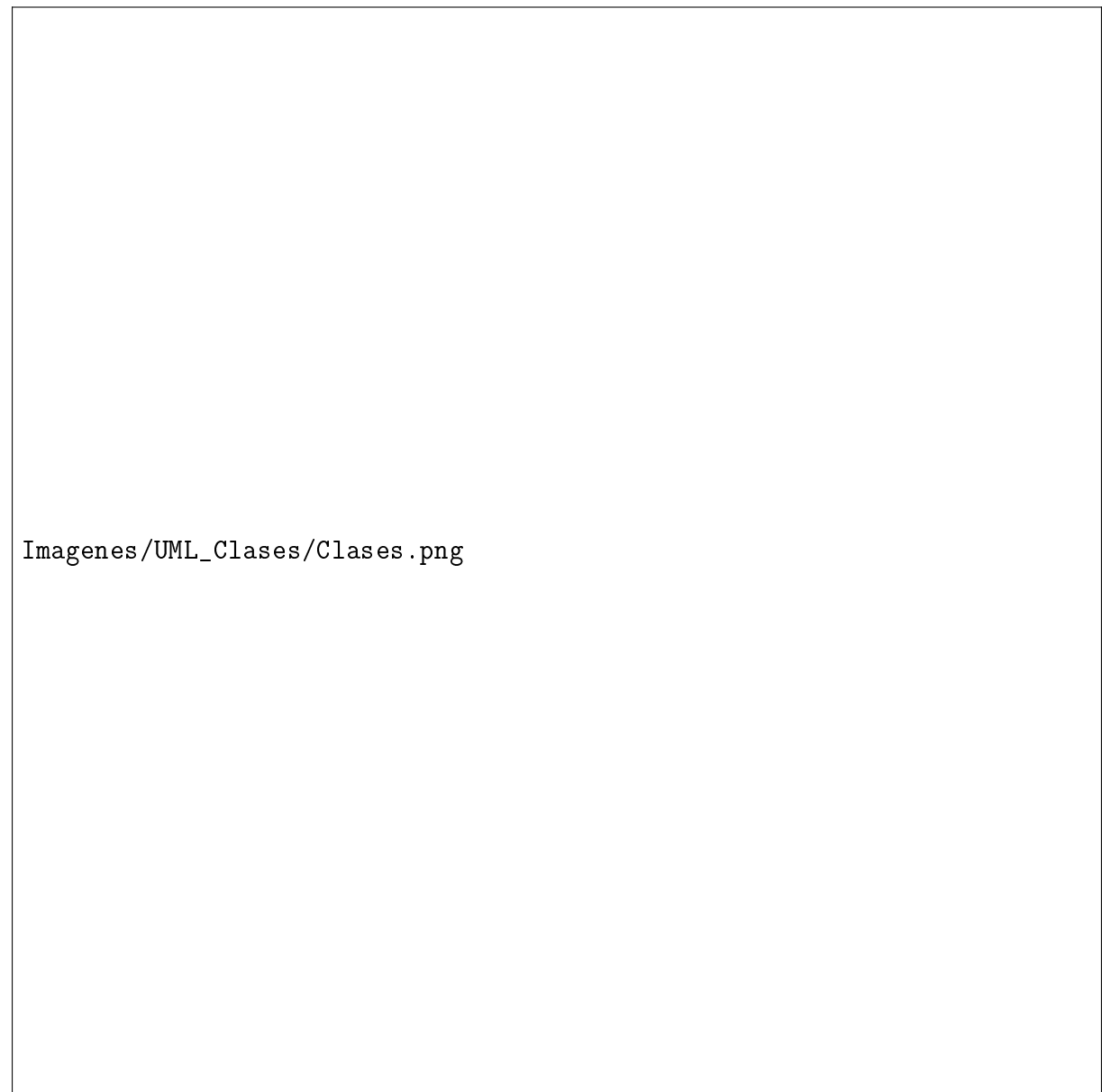
El **Cajero** (figura ??) tiene acceso a las funciones operativas del punto de venta: iniciar sesión, registrar ventas, seleccionar productos, consultar precios, generar tickets y finalizar transacciones. No tiene acceso a configuraciones administrativas ni a la gestión de usuarios o productos.

El **Administrador** (figura ??) posee control total del sistema, incluyendo la gestión de productos, usuarios, inventario y clientes (altas, bajas, modificaciones y consultas), así como la consulta de reportes y la supervisión de operaciones.

El **Cliente** (figura ??) interactúa con el sistema de forma indirecta a través del cajero: solicita productos, agrega artículos al carrito, realiza el pago, añade propina si lo desea y recibe el ticket de compra.

3.2.2. Diagrama de Clases

El diagrama de clases modela la estructura estática del sistema, mostrando las clases, sus atributos, métodos y las relaciones entre ellas. Representa la implementación de los principios de POO: la herencia entre **Usuario** y sus subclases, el encapsulamiento de los atributos, y las asociaciones entre las entidades del dominio.



Imagenes/UML_Clases/Clases.png

Figura 3.4: Diagrama de clases del sistema NexoPay

3.2.3. Diagramas de Secuencia

Los diagramas de secuencia ilustran el flujo de mensajes entre los objetos del sistema durante la ejecución de un caso de uso específico, mostrando el orden temporal de las interacciones.

Administrador



Figura 3.5: Diagrama de secuencia – Inicio de sesión (Administrador)



Imágenes/DiagramasDeSecuencia/Administrador/CerrarSesion.png

Figura 3.6: Diagrama de secuencia – Cerrar sesión (Administrador)



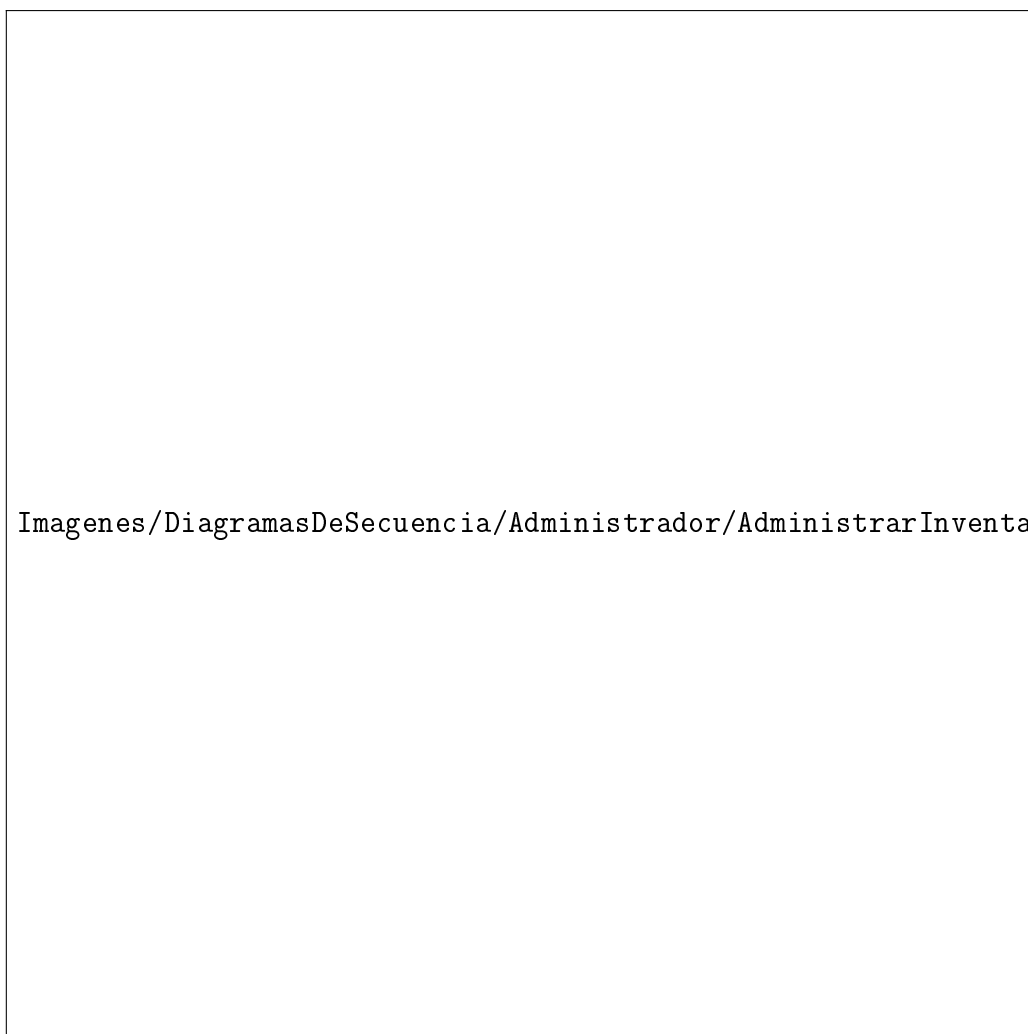
Figura 3.7: Diagrama de secuencia – Administrar productos (Administrador)



Figura 3.8: Diagrama de secuencia – Administrar usuarios (Administrador)



Figura 3.9: Diagrama de secuencia – Consultar reportes (Administrador)



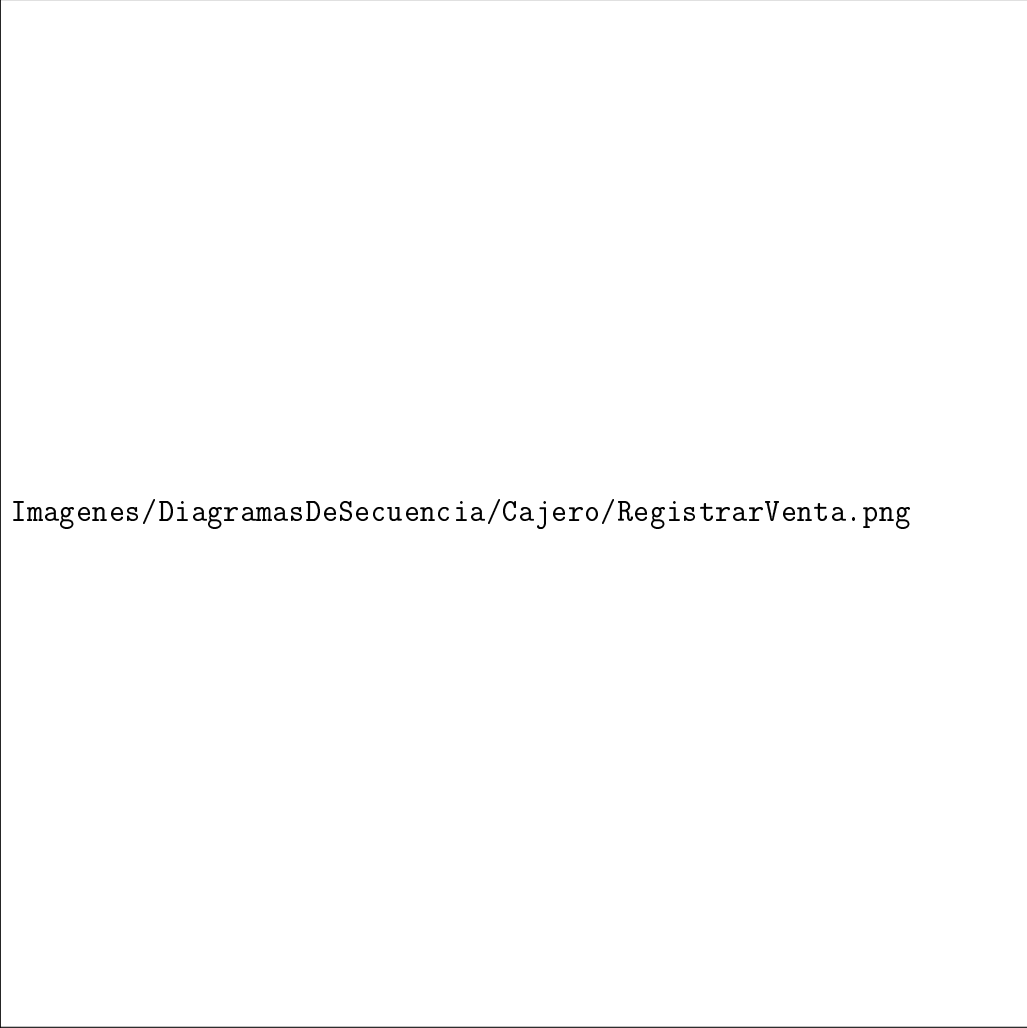
Imágenes/DiagramasDeSecuencia/Administrador/AdministrarInventario.png

Figura 3.10: Diagrama de secuencia – Administrar inventario (Administrador)

Cajero

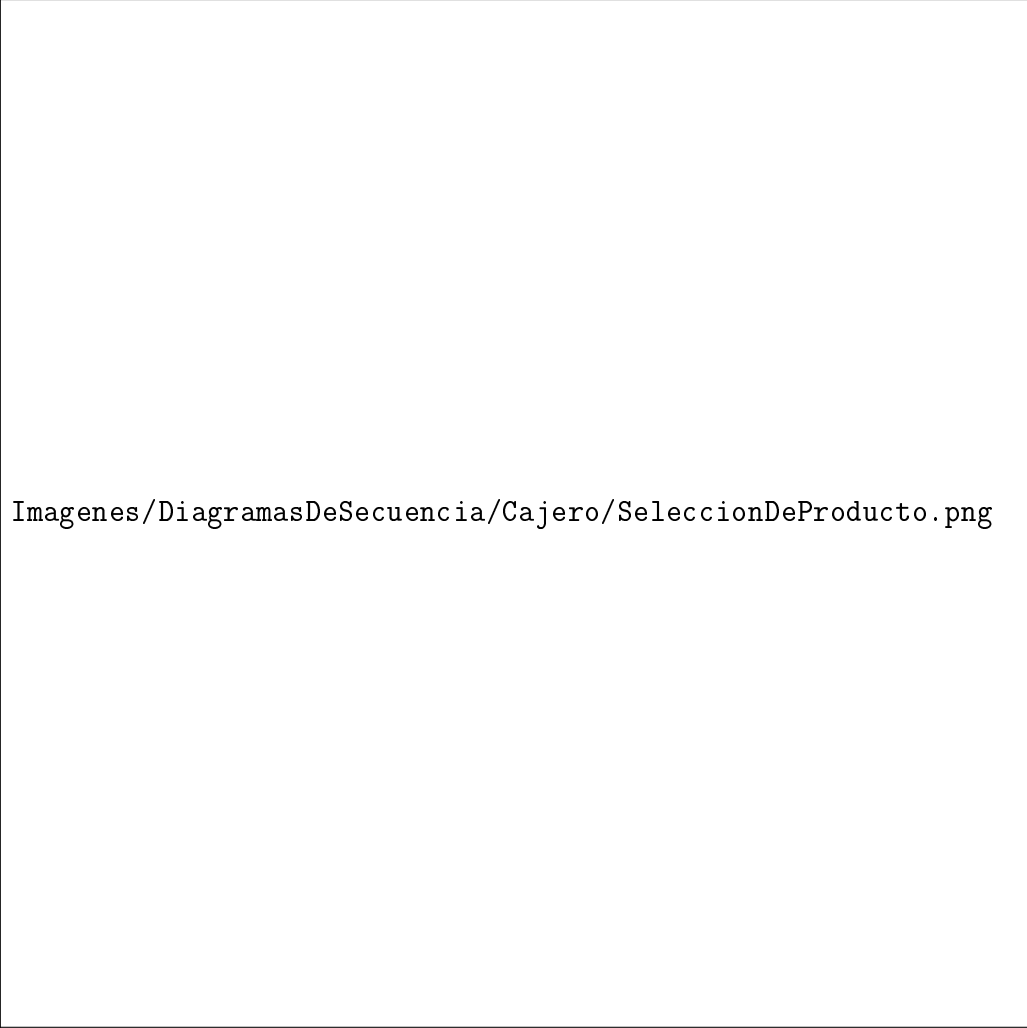


Figura 3.11: Diagrama de secuencia – Iniciar sesión (Cajero)



Imágenes/DiagramasDeSecuencia/Cajero/RegistrarVenta.png

Figura 3.12: Diagrama de secuencia – Registrar venta (Cajero)



Imágenes/DiagramasDeSecuencia/Cajero/SeleccionDeProducto.png

Figura 3.13: Diagrama de secuencia – Selección de productos (Cajero)



Figura 3.14: Diagrama de secuencia – Consultar precios de productos (Cajero)



Figura 3.15: Diagrama de secuencia – Generar ticket de venta (Cajero)

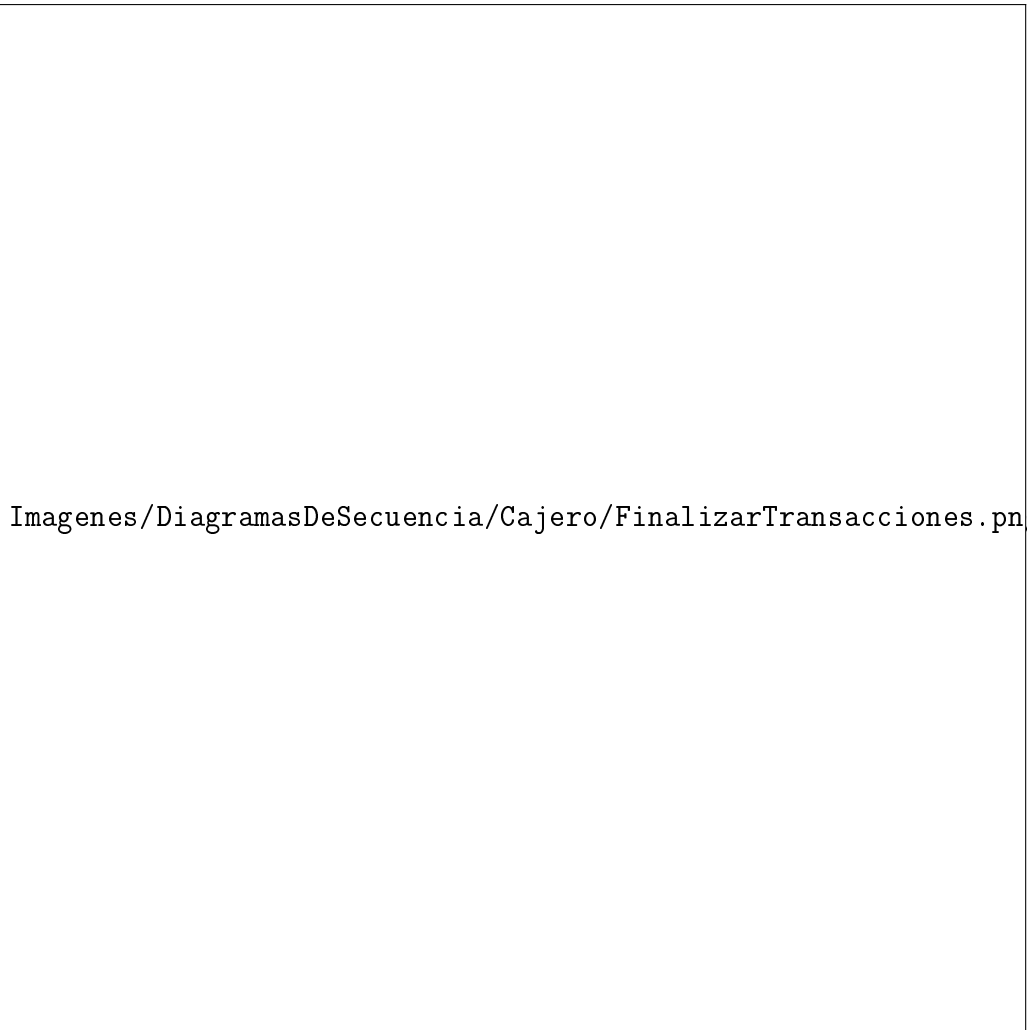


Figura 3.16: Diagrama de secuencia – Finalizar transacciones (Cajero)

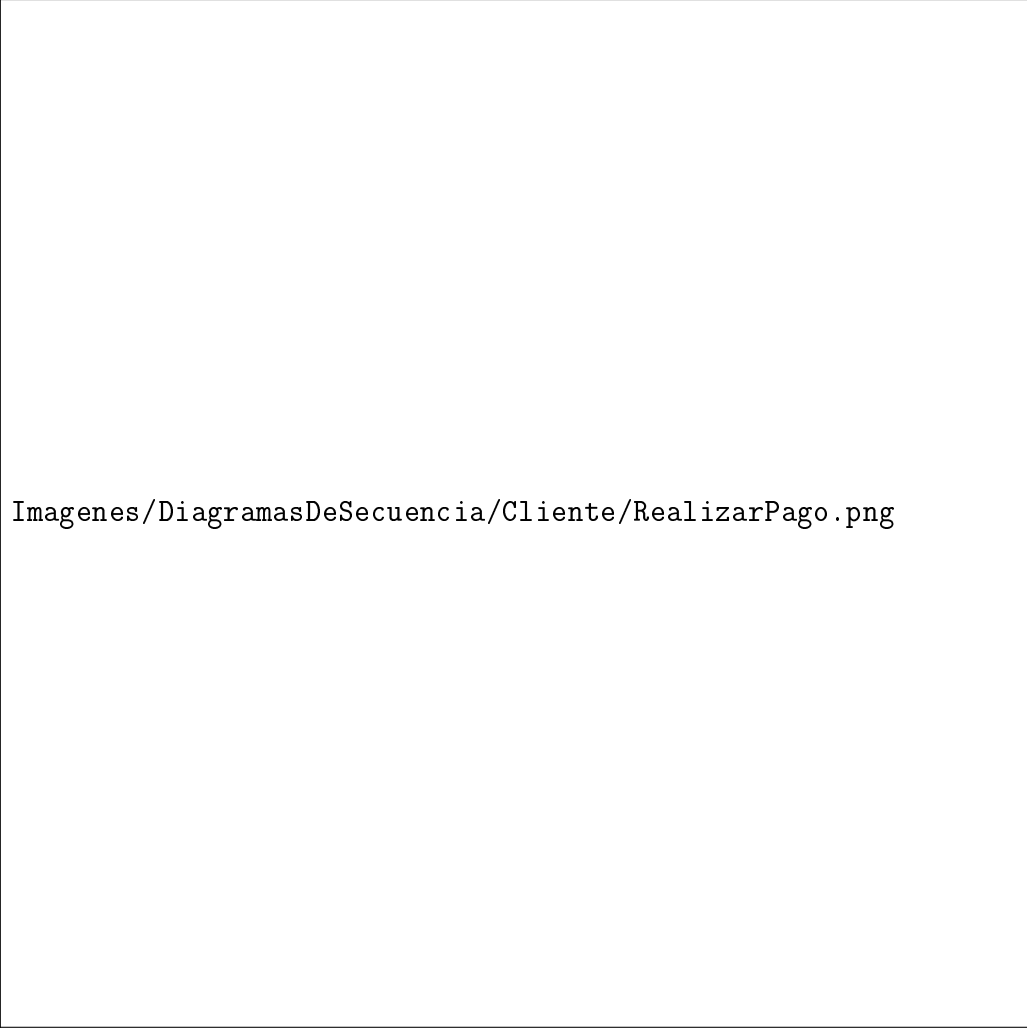
Cliente



Figura 3.17: Diagrama de secuencia – Solicitar productos (Cliente)

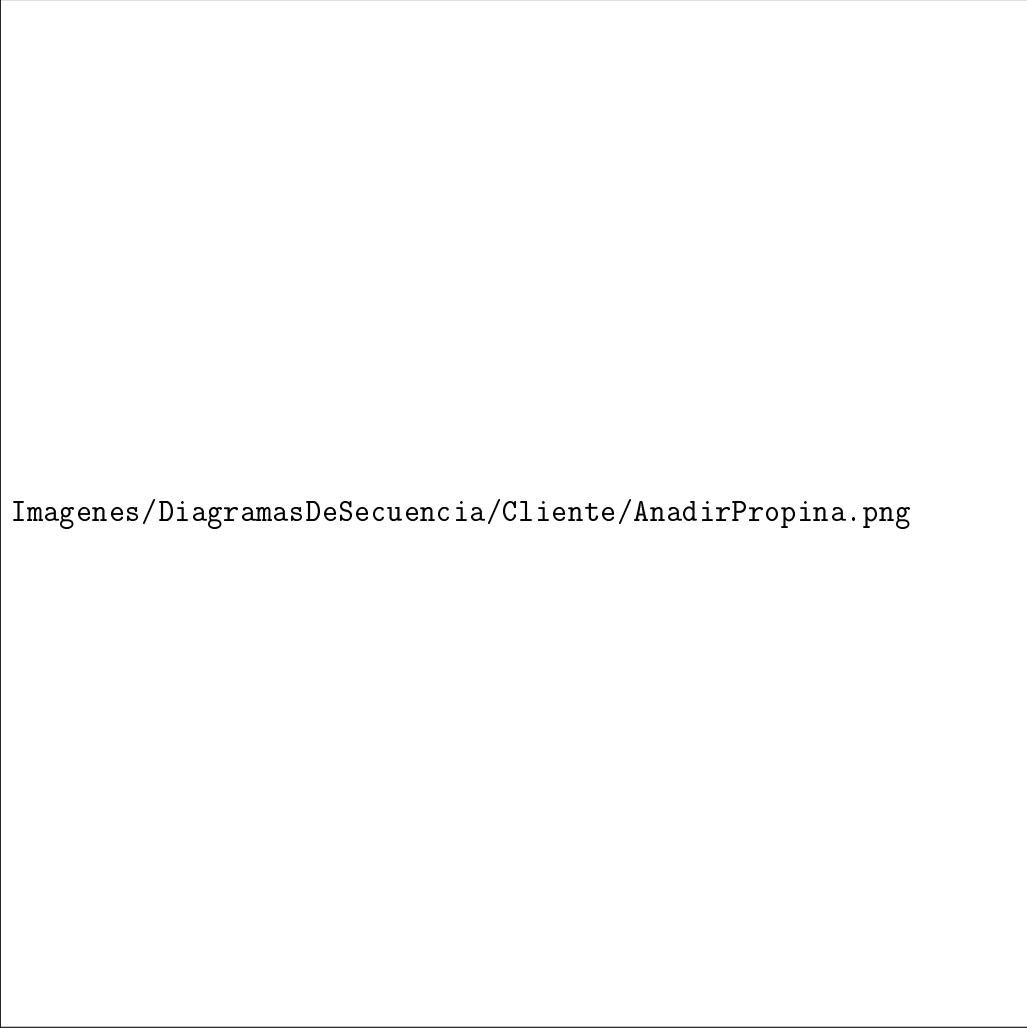


Figura 3.18: Diagrama de secuencia – Añadir productos al carrito (Cliente)



Imágenes/DiagramasDeSecuencia/Cliente/RealizarPago.png

Figura 3.19: Diagrama de secuencia – Realizar pago (Cliente)



Imágenes/DiagramasDeSecuencia/Cliente/AnadirPropina.png

Figura 3.20: Diagrama de secuencia – Añadir propina (Cliente)

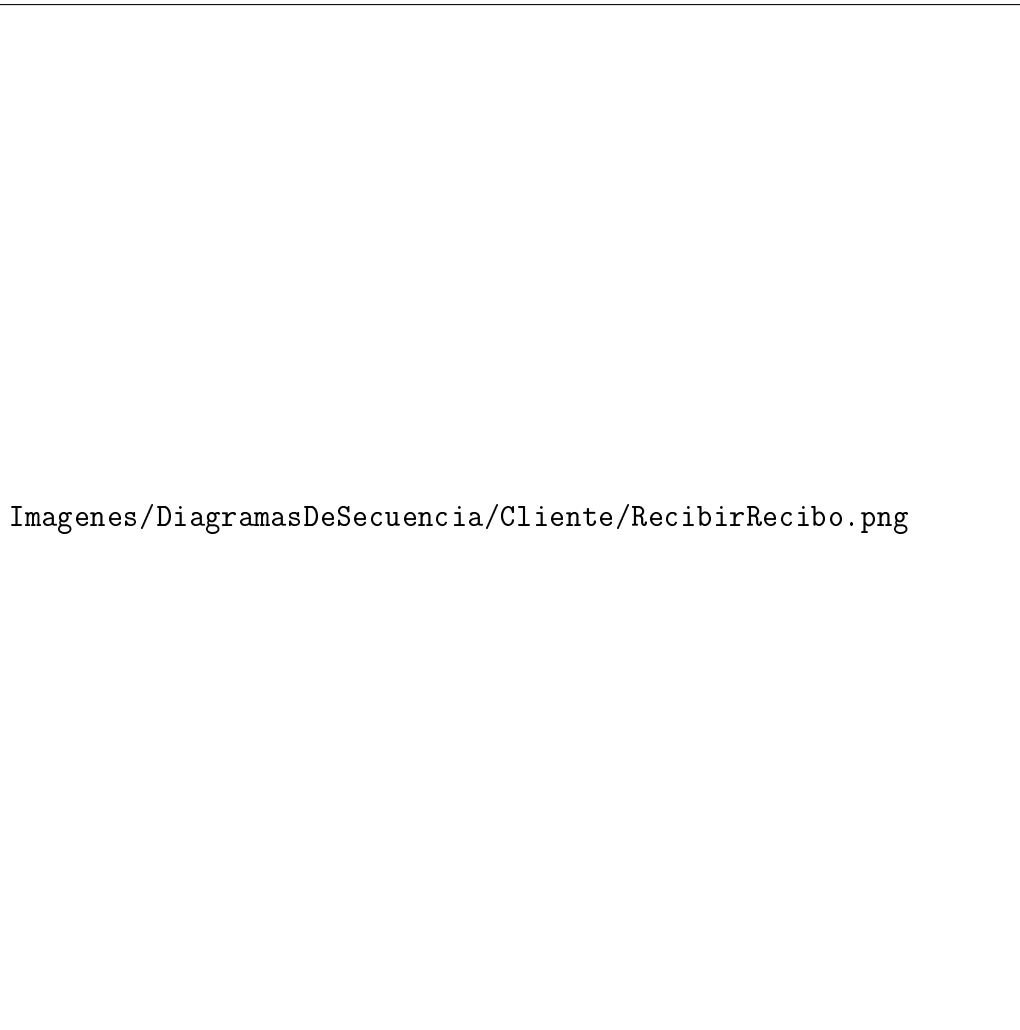


Figura 3.21: Diagrama de secuencia – Recibir ticket de compra (Cliente)

3.2.4. Diagramas de Estados

Los diagramas de estados modelan el ciclo de vida de los objetos principales del sistema, mostrando los estados posibles que pueden tomar y las transiciones entre ellos.



Imágenes/DiagramaEstados/Cajero_RealizarVenta.png

Figura 3.22: Diagrama de estados – Proceso de venta (Cajero)



Imágenes/DiagramaEstados/Admin_AltaProducto.png

Figura 3.23: Diagrama de estados – Alta de producto (Administrador)



Imágenes/DiagramaEstados/Admin_AltaUsuario.png

Figura 3.24: Diagrama de estados – Alta de usuario (Administrador)



Imágenes/DiagramaEstados/Cajero_EliminarProducto.png

Figura 3.25: Diagrama de estados – Eliminar producto (Cajero)



Figura 3.26: Diagrama de estados – Proceso de compra (Cliente)

3.2.5. Diagrama de Componentes

El diagrama de componentes muestra la organización modular del sistema, identificando los componentes de software y sus dependencias. NexoPay está organizado en los siguientes componentes: vistas (interfaz gráfica con Swing), controladores (lógica de control de flujo), servicios (lógica de negocio), modelos (clases del dominio), datos (acceso a base de datos) y utilidades (funciones de apoyo transversal).



Figura 3.27: Diagrama de componentes del sistema NexoPay

3.2.6. Diagrama de Despliegue

El diagrama de despliegue ilustra la infraestructura física sobre la que opera el sistema, especificando los nodos de hardware y la distribución del software entre ellos.



Figura 3.28: Diagrama de despliegue del sistema NexoPay

3.2.7. Diagrama Entidad-Relación

El diagrama entidad-relación (ER) modela la estructura de la base de datos del sistema, mostrando las entidades, sus atributos y las relaciones entre ellas con sus respectivas cardinalidades.

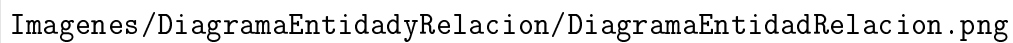
The image area is mostly blank, with a file path string 'Imagenes/DiagramaEntidadyRelacion/DiagramaEntidadRelacion.png' centered horizontally. This likely represents a missing or placeholder image for the ER diagram.

Figura 3.29: Diagrama entidad-relación de la base de datos de NexoPay

3.2.8. Explicación de los Diagramas UML y su Función en el Sistema

Cada diagrama elaborado cumple una función específica dentro del proceso de análisis y diseño:

Los **diagramas de casos de uso** definen el alcance funcional del sistema desde la perspectiva de los usuarios, sirviendo como base para la identificación de requerimientos y la planificación de las funcionalidades a implementar.

El **diagrama de clases** es el artefacto central del diseño orientado a objetos. Establece la estructura del código, guía la implementación y documenta las relaciones entre los componentes del software. En NexoPay, la jerarquía de clases refleja directamente los principios de POO aplicados.

Los **diagramas de secuencia** complementan al diagrama de clases mostrando el comportamiento dinámico: cómo los objetos colaboran entre sí en el tiempo para ejecutar un caso de uso. Son especialmente útiles para validar que el diseño es capaz de satisfacer los requerimientos funcionales.

Los **diagramas de estados** documentan el ciclo de vida de los objetos clave del sistema, como la venta (de “en proceso” a “finalizada”) o el producto (de “activo” a “eliminado”), ayudando a identificar las transiciones y validaciones necesarias en el código.

Los **diagramas de componentes y despliegue** ofrecen una vista arquitectónica del sistema, útil para planificar la implementación, la distribución del trabajo en equipo y la configuración del entorno de ejecución.

El **diagrama ER** guía la creación del esquema de la base de datos, asegurando que la estructura de almacenamiento sea coherente con el modelo de dominio del sistema.

3.3. Arquitecturas del Sistema

3.3.1. Diseño de la Arquitectura del Sistema

NexoPay adopta una **arquitectura monolítica en capas**, en la que todos los componentes del sistema conviven en un mismo ejecutable pero están organizados en capas con responsabilidades bien definidas y separadas. Esta organización interna sigue el patrón de diseño en capas, que divide el sistema en:

- **Capa de vistas (views):** Contiene las clases que implementan la interfaz gráfica con Swing. Es la única capa con la que interactúa directamente el usuario. No contiene lógica de negocio.
- **Capa de controladores (controllers):** Actúa como intermediaria entre las vistas y los servicios. Recibe los eventos generados por la interfaz, invoca los servicios

correspondientes y devuelve los resultados a la vista.

- **Capa de servicios (services):** Contiene la lógica de negocio del sistema: cálculo de totales, validación de stock, generación de tickets. Es independiente de la interfaz gráfica y del mecanismo de almacenamiento.
- **Capa de modelos (models):** Define las clases del dominio del problema: `Usuario`, `Producto`, `Venta`, `Cliente`, etc. Implementa los principios de POO mediante herencia, encapsulamiento y polimorfismo.
- **Capa de acceso a datos (data):** Gestiona la comunicación con la base de datos. Contiene las consultas SQL y la lógica de conexión, aislando al resto del sistema de los detalles de persistencia.
- **Capa de utilidades (utils):** Contiene funciones de apoyo transversal: validaciones de entrada, formateo de fechas y montos, generación de identificadores únicos.

3.3.2. Modelos Arquitectónicos Empleados

Arquitectura monolítica: El sistema se construye como una aplicación autónoma e integrada. Esta elección es coherente con la naturaleza del proyecto: una aplicación de escritorio para un comercio pequeño o mediano, donde la sencillez de instalación y la ausencia de dependencias de red son ventajas clave.

Patrón cliente-servidor (local): Dependiendo de la base de datos utilizada, el sistema puede operar en modo completamente local (SQLite, base de datos embebida) o conectándose a un servidor de base de datos MySQL en la misma red local. En el segundo caso, la aplicación actúa como cliente del servidor de base de datos.

Diseño en capas: Aunque el sistema es monolítico en su despliegue, internamente aplica una separación clara de responsabilidades que facilita el mantenimiento, las pruebas y la extensión del sistema.

3.3.3. Tecnologías y Herramientas Seleccionadas

Cuadro 3.4: Tecnologías y herramientas utilizadas en NexoPay

Tecnología / Herramienta	Categoría	Uso en el proyecto
Java (JDK 17+)	Lenguaje de programación	Implementación de toda la lógica del sistema con POO
Java Swing	Biblioteca de GUI	Construcción de la interfaz gráfica de escritorio
MySQL	Base de datos relacional	Almacenamiento persistente de datos en entornos con servidor
SQLite	Base de datos embebida	Alternativa de almacenamiento local sin servidor
JDBC	API de conectividad	Comunicación entre Java y la base de datos
Visual Studio Code	IDE	Desarrollo y depuración del código fuente
Draw.io / diagrams.net	Herramienta de diagramas	Elaboración de diagramas UML y ER
Git	Control de versiones	Gestión del código fuente y colaboración en equipo

3.4. Codiseño Hardware-Software

3.4.1. Relación entre Software y Hardware dentro del Sistema

Un sistema POS como NexoPay no opera de forma aislada; su funcionamiento efectivo depende de la correcta integración con componentes de hardware que facilitan la operación en el punto de venta. El software está diseñado para interactuar con estos componentes a través de interfaces estándar del sistema operativo y de Java.

La relación entre software y hardware en NexoPay puede entenderse en dos niveles: el hardware de cómputo que ejecuta el software (computadora de escritorio o laptop), y los periféricos especializados del punto de venta (impresora de tickets, lector de código de barras, visor de cliente y caja registradora).



Imagenes/Codiseno/SistemaPOS.png

Figura 3.30: Integración hardware-software del sistema NexoPay

3.4.2. Selección de Componentes de Hardware

Cuadro 3.5: Componentes de hardware del sistema POS

Componente	Función	Integración con el software
Computadora de escritorio / Laptop	Ejecutar la aplicación NexoPay	Plataforma principal; requiere JRE instalado
Monitor o pantalla	Mostrar la interfaz gráfica al cajero	Interfaz estándar del sistema operativo
Teclado y ratón	Entrada de datos por parte del cajero	Dispositivos de entrada estándar
Impresora de tickets (térmica)	Imprimir comprobantes de compra	Integración mediante la API de impresión de Java (<code>javax.print</code>)
Lector de código de barras	Escanear productos para agilizar su selección	Entrada de datos por teclado virtual (modo HID)
Visor de cliente (display)	Mostrar el total al cliente durante la venta	Comunicación por puerto serial o USB
Caja registradora / cajón de dinero	Almacenar el efectivo de las transacciones	Apertura activada por señal de la impresora

3.4.3. Estrategias de Integración Software-Hardware

La integración entre NexoPay y los componentes de hardware se realiza mediante las siguientes estrategias:

Abstracción de dispositivos: El software utiliza interfaces Java (`javax.print` para impresión, `javax.comm` o `JSerialComm` para puertos seriales) que abstraen los detalles específicos del hardware, permitiendo que el sistema funcione con distintos modelos de impresoras y lectores de código de barras sin modificar el código principal.

Entrada por teclado virtual (HID): Los lectores de código de barras modernos funcionan en modo HID (*Human Interface Device*), simulando la entrada de teclado. NexoPay aprovecha esta característica para capturar el código escaneado directamente en el campo de selección de productos, sin requerir controladores adicionales.

Generación de tickets por software: El sistema genera el contenido del ticket (productos, cantidades, precios, total, método de pago) en formato de texto plano y lo envía a la impresora térmica mediante el subsistema de impresión de Java, garantizando com-

patibilidad con la mayoría de los modelos del mercado.



Figura 3.31: Diagrama de decisión para la integración hardware-software

Capítulo 4

Implementación y Pruebas

4.1. Descripción de la Implementación

NexoPay se implementó como una aplicación de escritorio en Java, organizada en la siguiente estructura de paquetes que refleja la arquitectura en capas definida en el diseño:

- `models/` – Clases del dominio: `Usuario`, `Cajero`, `Administrador`, `Producto`, `Venta`, `DetalleVenta`, `Cliente`, `MetodoPago`.
- `services/` – Lógica de negocio: `AuthService`, `VentaService`, `ProductoService`, `InventarioService`, `ReporteService`.
- `controllers/` – Control de flujo entre vistas y servicios.
- `views/` – Pantallas Swing: login, menú principal, registro de ventas, gestión de productos, reportes.
- `data/` – Acceso a base de datos: `ConexionBD`, `UsuarioDAO`, `ProductoDAO`, `VentaDAO`.
- `utils/` – Validadores, formateadores y generador de tickets.

La herencia se aplica en la jerarquía de usuarios: la clase abstracta `Usuario` define los atributos y métodos comunes, mientras que `Cajero` y `Administrador` extienden esta clase añadiendo las funcionalidades específicas de cada rol. El polimorfismo se aprovecha al invocar el método `obtenerMenu()` sobre una referencia de tipo `Usuario`, que devuelve las opciones correspondientes al tipo concreto del objeto.

4.2. Consultas SQL de la Base de Datos

Las siguientes consultas SQL fueron implementadas en la capa de acceso a datos y corresponden a las operaciones más relevantes del sistema:

4.2.1. Consulta 1: Todos los usuarios y su tipo

```
1 SELECT Usuario.Nombre, tipousuario.NombreTipo
2 FROM Usuario
3 INNER JOIN tipousuario
4 ON Usuario.idTipoUsuario = tipousuario.idTipoUsuario;
```

Listing 4.1: Consulta todos los usuarios y su tipo de usuario



Imagenes/BaseDeDatos/todos_los_usuarios.png

Figura 4.1: Resultado – Todos los usuarios y su tipo

4.2.2. Consulta 2: Todos los productos del catálogo

```
1 SELECT * FROM producto;
```

Listing 4.2: Consulta todos los productos registrados

Imagenes/BaseDeDatos/tipos_de_usuarios.png

Figura 4.2: Resultado – Catálogo completo de productos

4.2.3. Consulta 3: Detalles de venta con productos correspondientes

```
1 SELECT DetalleVenta.idVenta, Producto.Nombre,  
2         DetalleVenta.Cantidad, DetalleVenta.SubTotal  
3 FROM DetalleVenta  
4 INNER JOIN Producto  
5 ON DetalleVenta.idProducto = Producto.idProducto;
```

Listing 4.3: Consulta detalles de venta y productos asociados



Figura 4.3: Resultado – Detalles de venta con productos

4.2.4. Consulta 4: Ventas de un cliente y cajero que lo atendió

```
1 SELECT Venta.idVenta, Cliente.Nombre, Usuario.Nombre
2 FROM Venta
3 INNER JOIN Cliente ON Venta.idCliente = Cliente.idCliente
4 INNER JOIN Usuario ON Venta.idUsuario = Usuario.idUsuario
5 WHERE Cliente.idCliente = 2;
```

Listing 4.4: Ventas de un cliente específico con cajero que lo atendió



Imagenes/BaseDeDatos/facturas_con_usuario.png

Figura 4.4: Resultado – Ventas del cliente 2 y cajero asignado

4.2.5. Consulta 5: Clientes atendidos por un cajero

```
1 SELECT Cliente.Nombre, Cliente.Apellido, Usuario.Nombre
2 FROM Venta
3 INNER JOIN Cliente ON Venta.idCliente = Cliente.idCliente
4 INNER JOIN Usuario ON Venta.idUsuario = Usuario.idUsuario
5 WHERE Usuario.idUsuario = 2;
```

Listing 4.5: Clientes atendidos por un cajero específico



Imagenes/BaseDeDatos/usuario_con_sus_facturas.png

Figura 4.5: Resultado – Clientes atendidos por el cajero 2

4.3. Estrategia de Pruebas y Validación de Requerimientos

La estrategia de pruebas de NexoPay se apoya en dos técnicas complementarias: pruebas de caja negra y tablas de decisión.

4.3.1. Pruebas de Caja Negra

Las pruebas de caja negra verifican el comportamiento del sistema a partir de entradas y salidas esperadas, sin considerar la implementación interna. Se definen los escenarios de prueba a partir de los flujos principales y los casos de error de cada funcionalidad.

Cuadro 4.1: Plan de pruebas de caja negra – Sistema NexoPay

ID	Escenario / Caso de Prueba	Resultado Esperado
1	Iniciar sesión con usuario “admin” y contraseña correcta	Ingreso exitoso al panel de administración
2	Iniciar sesión con usuario “admin” y contraseña incorrecta	Mensaje de error: “Contraseña incorrecta”
3	Registrar un producto con precio de \$0.00	Mensaje de error: “El precio debe ser mayor a 0”
4	Registrar un producto con nombre vacío	Mensaje de error: “El nombre es obligatorio”
5	Agregar 10 unidades al carrito cuando solo hay 5 en stock	Mensaje de error: “Stock insuficiente”
6	Realizar una venta y pagar con un método no registrado	Transacción rechazada
7	Consultar un producto por un ID que no existe	Mensaje: “Producto no encontrado”
8	Finalizar venta con carrito lleno y pago en efectivo	Ticket generado y stock actualizado
9	Eliminar un usuario administrador del sistema	Confirmación de baja exitosa
10	Modificar stock con valor negativo que deja el total bajo cero	Error: “El ajuste dejaría el stock en negativo”

4.3.2. Tablas de Decisión

Las tablas de decisión modelan las combinaciones posibles de condiciones de entrada y los resultados esperados para cada combinación, asegurando la cobertura de todos los escenarios relevantes.

Cuadro 4.2: Tabla de decisión – Inicio de sesión

Condición / Resultado	R1	R2	R3	R4
El usuario existe en la BD	V	V	F	F
La contraseña es correcta	V	F	V	F
Permitir acceso al sistema	X			
Mostrar error: Credenciales inválidas		X	X	X

Cuadro 4.3: Tabla de decisión – Finalizar venta

Condición / Resultado	R1	R2	R3	R4
Hay productos en el carrito	V	V	F	F
Método de pago es válido	V	F	V	F
Generar ticket y registrar venta	X			
Mostrar error: Carrito vacío o pago inválido		X	X	X

Capítulo 5

Conclusiones

5.1. Reflexión sobre el Desarrollo del Proyecto

El desarrollo de NexoPay representó un ejercicio integral de ingeniería de software, en el que los conceptos teóricos de la Programación Orientada a Objetos se tradujeron en decisiones concretas de diseño e implementación. La jerarquía de clases de usuarios, la separación del sistema en capas con responsabilidades bien definidas y la integración con una base de datos relacional son ejemplos tangibles de cómo los principios de POO facilitan la construcción de software organizado, legible y mantenible.

El proceso de análisis previo al desarrollo resultó fundamental: los diagramas UML elaborados permitieron anticipar problemas de diseño, definir con claridad las responsabilidades de cada componente y establecer un mapa de trabajo compartido para el equipo. Esta experiencia refuerza la importancia de dedicar tiempo al diseño antes de comenzar a codificar.

La integración de múltiples áreas del conocimiento, incluyendo bases de datos, interfaces gráficas, lógica de negocio y principios de diseño, demostró que el desarrollo de software real es una actividad multidisciplinaria que requiere no solo habilidades técnicas, sino también capacidad de planificación, comunicación y trabajo en equipo.

5.2. Lecciones Aprendidas

- **El diseño previo ahorra tiempo de desarrollo:** Los diagramas UML elaborados antes de comenzar a codificar actuaron como una guía clara que redujo la ambigüedad y el retrabajo durante la implementación.
- **La separación de responsabilidades facilita el mantenimiento:** Organizar el código en capas (vistas, controladores, servicios, modelos, datos) hizo que las modificaciones en una capa raramente afectaran a las demás, simplificando las correcciones y mejoras.
- **Los nombres descriptivos son más importantes de lo que parecen:** Invertir tiempo en elegir nombres claros para clases, métodos y variables redujo significativamente el tiempo necesario para comprender el código al revisarlo días después de haberlo escrito.
- **Las pruebas deben planificarse desde el inicio:** Definir los casos de prueba junto con los requerimientos permitió detectar casos límite (stock negativo, carrito vacío, credenciales inválidas) que de otro modo podrían haberse pasado por alto hasta etapas tardías del desarrollo.
- **La base de datos requiere diseño cuidadoso:** Las relaciones entre tablas, las restricciones de integridad referencial y la elección de los tipos de datos tuvieron un impacto directo en la facilidad de implementación de las consultas y en la consistencia de los datos.

5.3. Posibles Mejoras Futuras

Con miras a versiones futuras del sistema, se identifican las siguientes mejoras que ampliarían sus capacidades y su robustez:

- **Módulo de devoluciones:** Implementar la funcionalidad de procesar devoluciones de productos, con actualización automática del inventario y registro de la operación en el historial de ventas.

- **Gestión de descuentos y promociones:** Añadir soporte para descuentos por producto, por categoría o por monto total de la venta, incluyendo la configuración de vigencias y condiciones.
- **Reportes avanzados con gráficas:** Enriquecer el módulo de reportes con visualizaciones gráficas (barras, líneas) que faciliten el análisis de tendencias de ventas a lo largo del tiempo.
- **Soporte multi-sucursal:** Extender la arquitectura para soportar múltiples puntos de venta conectados a una base de datos centralizada, con sincronización de inventario en tiempo real.
- **Exportación de reportes a PDF:** Permitir la exportación de los reportes de ventas en formato PDF para su archivo y distribución.
- **Notificaciones de stock bajo:** Implementar alertas automáticas cuando el stock de un producto caiga por debajo de un umbral configurable, facilitando la gestión de reabastecimiento.
- **Migración a arquitectura cliente-servidor web:** En una versión más avanzada, migrar la interfaz de Swing a una interfaz web con JavaFX o tecnologías web estándar, permitiendo el acceso al sistema desde múltiples dispositivos.

Capítulo 6

Bibliografía

Bibliografía

- [1] K. C. Laudon and J. P. Laudon, *Sistemas de información gerencial*, 12^a ed. Pearson Educación, 2012.
- [2] P. J. Deitel and H. M. Deitel, *Java: How to Program*, 9th ed. Pearson College Division, 2012.
- [3] Oracle, “Java Documentation – Get Started,” Oracle Help Center, Jan. 31, 2023. [Online]. Available: <https://docs.oracle.com/en/java/>
- [4] T. Babic, “20 ejemplos de consultas SQL básicas para principiantes: Una visión completa,” *LearnSQL.es*, Sep. 20, 2023. [Online]. Available: <https://learnsql.es/blog/20-ejemplos-de-consultas-sql-basicas-para-principiantes-una-vision-completa/>
- [5] J. M. Alarcón, “TUTORIAL SQL #3: Consultas SELECT multi-tabla básicas – JOIN,” *campusMVP.es*, Nov. 08, 2021. [Online]. Available: <https://www.campusmvp.es/recursos/post/Fundamentos-de-SQL-Consultas-SELECT-multi-tabla-JOIN.aspx>
- [6] I. Sommerville, *Ingeniería del Software*, 9^a ed. Pearson Educación, 2011.
- [7] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7^a ed. McGraw-Hill, 2010.
- [8] J. Rumbaugh, I. Jacobson, and G. Booch, *El Lenguaje Unificado de Modelado. Manual de Referencia*, 2^a ed. Pearson Educación, 2004.